Lesson 3 Introduction to the Servo Fine-tuning Function

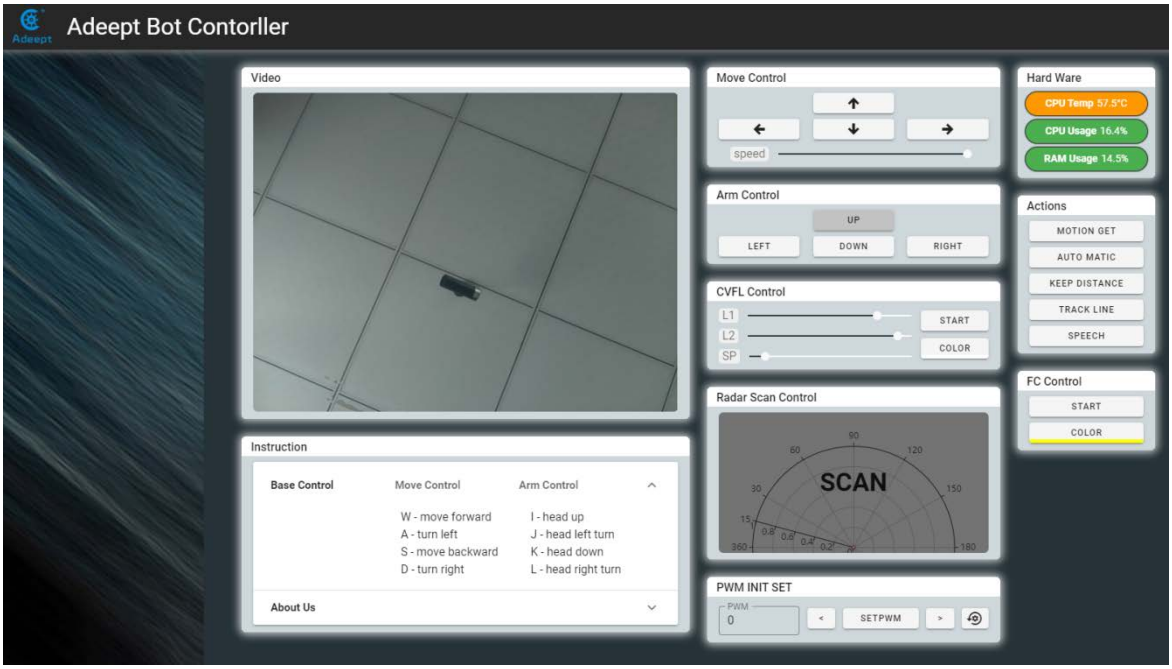
3.1 Function introduction

Available to fine-tuning the default angle of the servo and correct the angle error caused by the servo installation. For the crawling robots, fine-tuning the default angle of the servo will improve their walking smoothness. For robot products with manipulators, it helps to adjust the initial angle of the manipulator chuck for correcting the error caused by the servo installation.

3.2 Run the servo fine-tuning program

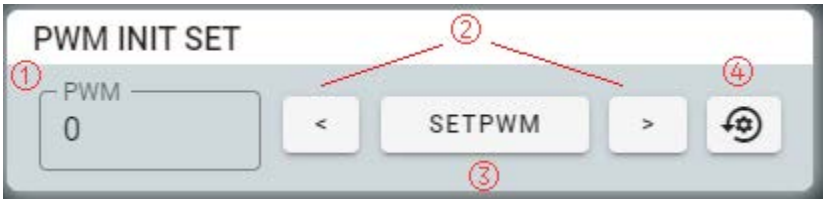
1. ©Start up Raspberry Pi robot for about 30s-60s.

2. When PiCar-b is started up, enter the IP address of your Raspberry Pi on the Google browser of the mobile phone or computer, and access Port 5000, for example: 192.168.3.44:5000. Then the web controller will be displayed on the browser.



3. The module "PWM INIT SET" is the operating area of the fine-tuning function of the servo. (It is recommended to restart PiCar-b before fine-tuning the servo)

4. Introduction to the servo fine-tuning module



- "PWM": for selecting the servo number and the No. 0 servo is selected in the figure. Click the box to enter the number or click the buttons of up and down in the box to select the servo number that needs adjusting.

- "<": Adjust the selected servo to rotate counterclockwise slightly.
- “>”: Adjust the selected servo to rotate clockwise slightly.
- "SETPWM": After adjusting the servo, save the current servo angle as the default value.
- "@": Initialize the default angles of all servos to factory setting.

5. Select the servo number to be adjusted. The servo number is the interface number of the servo connected to Robot HAT. For example, when the servo cable is connected to the Servo 1 interface, the servo number is 1.

6. After selecting the servo to be adjusted, clicking "<" and ">" to adjust the servo to a right position, and then click "SETPWM" to save.

3.3 The main code program

Please see the complete code in findline.py.

Import dependency and initialize

```
from __future__ import division
import time
import RPi.GPIO as GPIO
import sys
import Adafruit_PCA9685
import threading

import random
'''''
change this form 1 to -1 to reverse servos
'''
pwm = Adafruit_PCA9685.PCA9685()
pwm.set_pwm_freq(50)

init_pwm0 = 300
init_pwm1 = 300
init_pwm2 = 300
init_pwm3 = 300

init_pwm4 = 300
init_pwm5 = 300
init_pwm6 = 300
init_pwm7 = 300

init_pwm8 = 300
init_pwm9 = 300
init_pwm10 = 300
init_pwm11 = 300

init_pwm12 = 300
init_pwm13 = 300
init_pwm14 = 300
init_pwm15 = 300
```

Servo fine-tuning function

```
class ServoCtrl(threading.Thread):

    def __init__(self, *args, **kwargs):
        self.sc_direction = [1,1,1,1, 1,1,1,1, 1,1,1,1, 1,1,1,1]
        self.initPos = [init_pwm0,init_pwm1,init_pwm2,init_pwm3,
                        init_pwm4,init_pwm5,init_pwm6,init_pwm7,
                        init_pwm8,init_pwm9,init_pwm10,init_pwm11,
                        init_pwm12,init_pwm13,init_pwm14,init_pwm15]
        self.goalPos = [300,300,300,300, 300,300,300,300 ,300,300,300,300 ,300,300,300,300]
        self.nowPos  = [300,300,300,300, 300,300,300,300 ,300,300,300,300 ,300,300,300,300]
        self.bufferPos  = [300.0,300.0,300.0,300.0, 300.0,300.0,300.0,300.0 ,300.0,300.0,300.0,300.0 ,300.0,300.0,300.0,300.0]
        self.lastPos = [300,300,300,300, 300,300,300,300 ,300,300,300,300 ,300,300,300,300]
        self.ingGoal = [300,300,300,300, 300,300,300,300 ,300,300,300,300 ,300,300,300,300]
        self.maxPos  = [560,560,560,560, 560,560,560,560 ,560,560,560,560 ,560,560,560,560]
        self.minPos  = [100,100,100,100, 100,100,100,100 ,100,100,100,100 ,100,100,100,100]
        self.scSpeed = [0,0,0,0, 0,0,0,0 ,0,0,0,0 ,0,0,0,0]

        self.ctrlRangeMax = 560
        self.ctrlRangeMin = 100
        self.angleRange = 180

        '''''
        scMode: 'init' 'auto' 'certain' 'quick' 'wiggle'
        '''
        self.scMode = 'auto'
        self.scTime = 2.0
        self.scSteps = 30

        self.scDelay = 0.037
        self.scMoveTime = 0.037

        self.goalUpdate = 0
        self.wiggleID = 0
        self.wiggleDirection = 1

```

```
        super(ServoCtrl, self).__init__(*args, **kwargs)
        self.__flag = threading.Event()
        self.__flag.clear()

    def initConfig(self, ID, initInput, moveTo):
        if initInput > self.minPos[ID] and initInput < self.maxPos[ID]:
            self.initPos[ID] = initInput
            if moveTo:
                pwm.set_pwm(ID,0,self.initPos[ID])
        else:
            print('initPos Value Error.')
```